

MC613 – Relatório 3

Projeto: Controlador DRAM

Rodrigo Banin Ferraz de Camargo (238257)

Maio de 2026

Sumário

1	Planejamento	2
1.1	Diagramas da etapa inicial	2
2	Estrutura do Código	5
2.1	Arquivos principais	5
2.2	Arquivos de validação	5
2.3	Descrição dos componentes	5
3	Implementação	5
3.1	Decisões centrais de implementação	5
3.2	Trechos essenciais de código	6
3.2.1	Buffer de comando no <code>reader</code>	6
3.2.2	Arbitragem no estado <code>READY</code> do controlador	6
3.2.3	Escrita com habilitação do barramento de dados	6
3.2.4	Leitura automática pós-escrita no <code>dram_iface</code>	7
3.2.5	Integração ativa no top-level	7
4	Validação (Testes)	7
4.1	Teste de componente: <code>tb_dram_iface.vhd</code>	7
4.2	Teste de componente: <code>tb_reader.vhd</code>	8
4.3	Teste de componente: <code>tb_dram_controller.vhd</code>	8
4.4	Teste de integração: <code>tb_dram_system.vhd</code>	8
5	Análise de Execução	9
5.1	Temporização da FSM	9
5.2	Controle do barramento <code>DRAM_DQ</code>	9
5.3	Integração interface-controlador	9

1 Planejamento

O planejamento foi orientado pelos requisitos da entrega final: implementar um controlador DRAM funcional, integrar com a interface da placa e validar por simulação os fluxos obrigatórios (INIT, READ, WRITE, REFRESH e WRITE seguido de READ automático).

A arquitetura foi dividida em camadas para reduzir complexidade:

- **dram_iface**: camada de interação com usuário (SW/KEY/HEX), que transforma ações em requisições de leitura/escrita.
- **dram_controller**: camada de protocolo DRAM, responsável por comandos e temporização.

Essa separação permitiu evolução incremental: primeiro validar emissão de requisições da interface, depois validar temporização do controlador, e por fim validar integração entre os dois.

1.1 Diagramas da etapa inicial

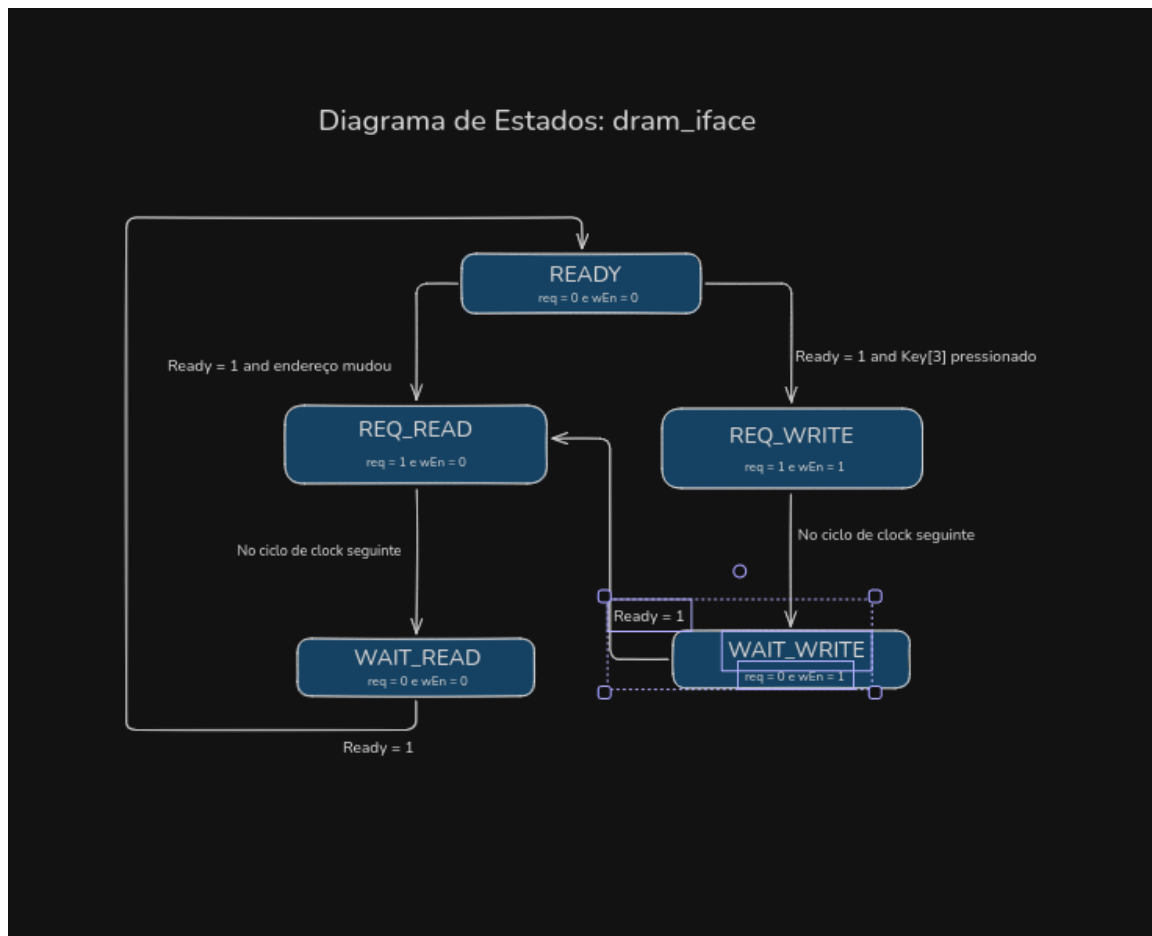


Figura 1: Diagrama de estados do módulo **dram_iface**.

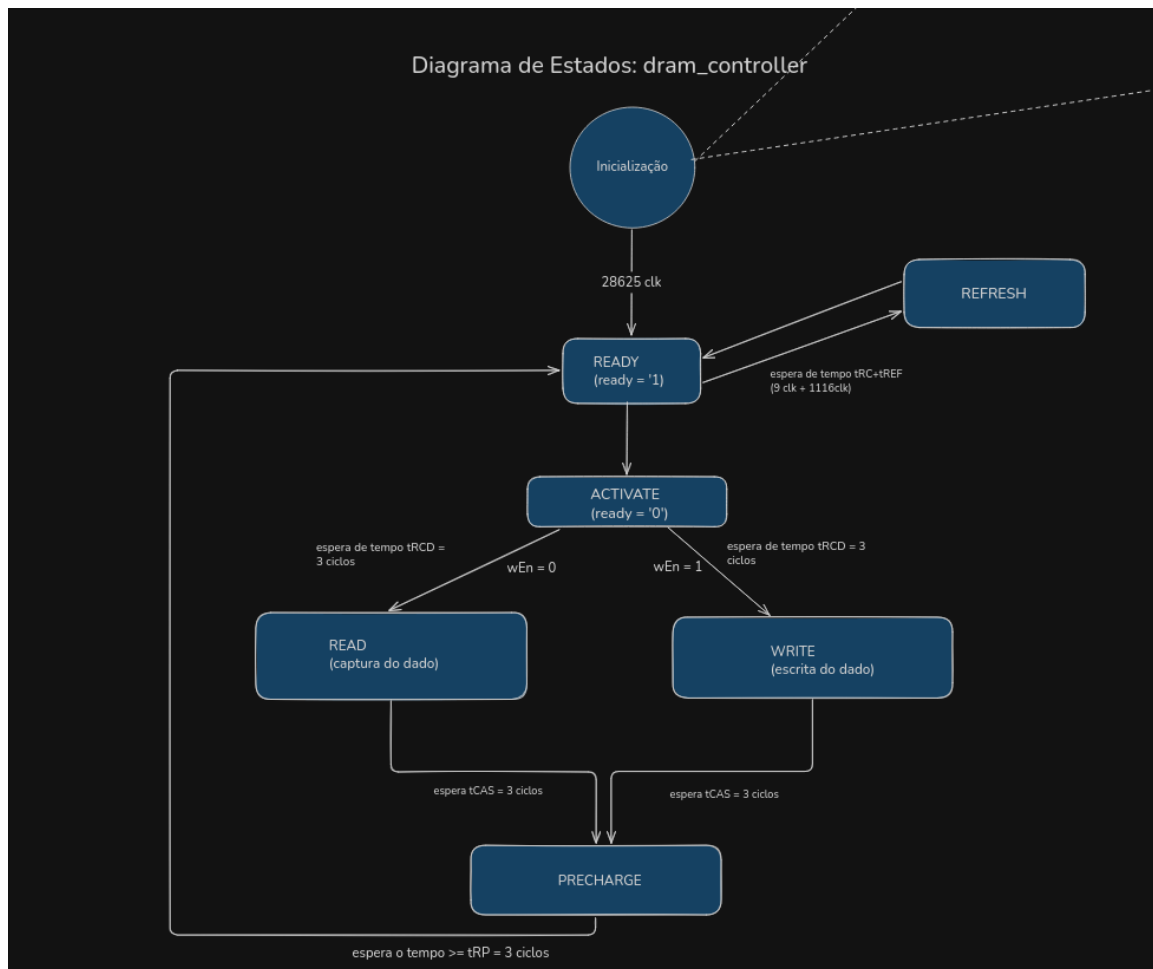


Figura 2: Diagrama de estados do módulo `dram_controller`.

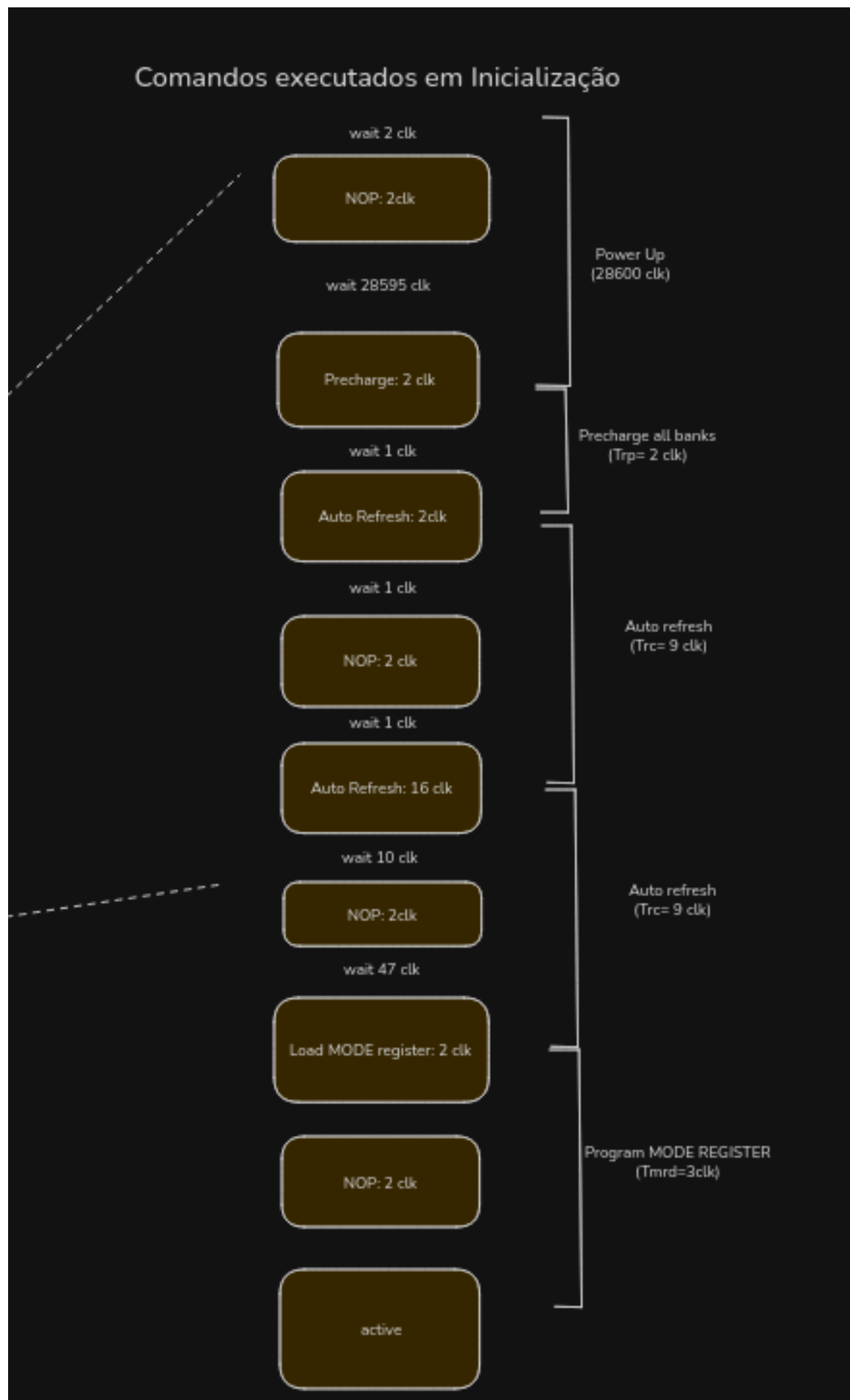


Figura 3: Sequência de inicialização da SDRAM.

2 Estrutura do Código

2.1 Arquivos principais

- `dram_iface.vhd`
- `reader.vhd`
- `dram_controller.vhd`
- `dram_top_level.vhd`

2.2 Arquivos de validação

- `tb_dram_iface.vhd`: valida apenas a FSM de interface.
- `tb_reader.vhd`: valida o buffer de comandos entre interface e controlador.
- `tb_dram_controller.vhd`: valida apenas o protocolo e temporização do controlador.
- `tb_dram_system.vhd`: valida integração `dram_iface` + `dram_controller` com modelo de memória.

2.3 Descrição dos componentes

dram_iface: responsável por gerar `req/wEn`, detectar escrita por borda de `KEY[3]`, disparar leitura por mudança de endereço e fazer leitura automática pós-escrita.

reader: responsável por capturar uma requisição de comando, manter endereço/-dado/mode de escrita estáveis e liberar a FSM principal apenas quando o controlador confirma consumo por `cmd_ack`.

dram_controller: responsável por consumir comandos bufferizados pelo **reader**, executar comandos SDRAM (ACTIVATE, READ, WRITE, PRECHARGE, AUTO REFRESH, MRS), aplicar tempos de espera e controlar o barramento `DRAM_DQ` bidirecional.

dram_top_level: responsável pela integração com PLL, interface de usuário e pinos físicos da SDRAM na DE1-SoC.

3 Implementação

3.1 Decisões centrais de implementação

As decisões técnicas mais importantes foram:

- uso de FSM refinada com estados intermediários de espera para cada tempo crítico;
- uso do **reader** como registrador de comando entre interface e FSM de SDRAM;
- codificação explícita de comandos DRAM por constantes;
- parametrização de temporização por genéricos (reuso de RTL com ajuste de latência);
- controle explícito de direção do barramento `DRAM_DQ` em escrita.

3.2 Trechos essenciais de código

3.2.1 Buffer de comando no reader

Listing 1: Captura de comando ate o acknowledge do controlador.

```
1  if cmd_ack = '1' then
2      pending_reg <= '0';
3  end if;
4
5  if req_in = '1' and (pending_reg = '0' or cmd_ack = '1') then
6      pending_reg <= '1';
7      wEn_reg      <= wEn_in;
8      addr_reg     <= addr_in;
9      data_reg     <= data_in;
10 end if;
```

3.2.2 Arbitragem no estado READY do controlador

Listing 2: Refresh tem prioridade antes de nova requisição externa.

```
1  when ST_READY =>
2      if refresh_pending = '1' then
3          refresh_pending <= '0';
4          state <= ST_REFRESH_CMD;
5      elsif buffered_req = '1' then
6          cmd_ack <= '1';
7          if buffered_wEn = '1' then
8              state <= ST_WRITE_ACT;
9          else
10             state <= ST_READ_ACT;
11         end if;
12     end if;
```

3.2.3 Escrita com habilitação do barramento de dados

Listing 3: No WRITE, o controlador dirige DRAM_DQ apenas no ciclo correto.

```
1  when ST_WRITE_TRCD =>
2      if wait_counter = 0 then
3          ras_n_reg <= CMD_WRITE_RAS_N;
4          cas_n_reg <= CMD_WRITE_CAS_N;
5          we_n_reg  <= CMD_WRITE_WE_N;
6          dq_out(7 downto 0) <= latched_wdata;
7          dq_out(15 downto 8) <= (others => '0');
8          dq_oe <= '1';
9          state <= ST_WRITE_TDPL;
10 end if;
```

3.2.4 Leitura automática pós-escrita no dram_iface

Listing 4: Ao fim da escrita, o próprio iface dispara READ de confirmação.

```
1 when WAIT_WRITE_ST =>
2   if ready = '1' then
3     req_reg <= '1';
4     wen_reg <= '0';
5     state   <= REQ_READ_ST;
6   end if;
```

3.2.5 Integração ativa no top-level

Listing 5: Instância real do controlador no projeto sintetizável.

```
1 controller_i : entity work.dram_controller
2   port map (
3     clk => pll_clk, rst => rst,
4     address => address, write_data => write_data, read_data =>
5       read_data,
6     req => req, wEn => wEn, ready => ready,
7     DRAM_ADDR => DRAM_ADDR, DRAM_BA => DRAM_BA, DRAM_DQ => DRAM_DQ,
8     DRAM_CS_N => DRAM_CS_N, DRAM_RAS_N => DRAM_RAS_N,
9     DRAM_CAS_N => DRAM_CAS_N, DRAM_WE_N => DRAM_WE_N
10  );
```

4 Validação (Testes)

A estratégia de validação foi dividida em quatro níveis: componente de interface, buffer de comandos, componente de controle e integração de sistema.

4.1 Teste de componente: tb_dram_iface.vhd

Objetivo: validar a FSM de interface isoladamente.

Cenários cobertos:

- leitura inicial automática;
- leitura ao detectar mudança de endereço;
- escrita ao detectar borda de descida em KEY[3];
- leitura automática após conclusão da escrita.

Critério de aprovação: asserts verificam combinações corretas de req/wEn em cada transição esperada.

4.2 Teste de componente: `tb_reader.vhd`

Objetivo: validar o armazenamento temporário de comandos entre a interface e o controlador.

Cenários cobertos:

- captura de requisição com endereço, dado e modo de escrita;
- manutenção do comando pendente até `cmd_ack`;
- limpeza após `acknowledge`;
- captura de novo comando no mesmo ciclo em que o anterior é reconhecido.

Critério de aprovação: asserts verificam `req_pending`, `wEn_out`, `addr_out` e `data_out` antes e depois do `acknowledge`.

4.3 Teste de componente: `tb_dram_controller.vhd`

Objetivo: validar protocolo DRAM e temporização sem dependência da interface de usuário.

Cenários cobertos:

- inicialização completa até `ready`;
- escrita unitária e leitura de retorno no mesmo endereço;
- refresh automático por temporizador interno.

Critério de aprovação: asserts confirmam fim da inicialização, consistência do dado lido após escrita e ocorrência de refresh automático.

4.4 Teste de integração: `tb_dram_system.vhd`

Objetivo: validar o comportamento completo do sistema no fluxo real `iface` → `reader` → `controller` → DRAM.

Cenários cobertos (exigidos no enunciado):

- INIT: sequência de inicialização e subida de `ready`;
- READ: `ACTIVATE/READ` com latência;
- WRITE: `ACTIVATE/WRITE/PRECHARGE` com espera;
- REFRESH: emissão automática e retorno ao `READY`;
- WRITE seguido de READ automático.

Critério de aprovação: ordem de comandos, intervalos de temporização e valores de dados são verificados por asserts.

5 Análise de Execução

5.1 Temporização da FSM

O ponto mais crítico do projeto foi garantir que cada comando DRAM fosse emitido com espaçamento correto. A solução foi usar estados intermediários de espera dedicados, evitando compressão indevida entre ACTIVATE, READ/WRITE, PRECHARGE e REFRESH.

5.2 Controle do barramento DRAM_DQ

Outro ponto central foi o barramento bidirecional de dados. A separação entre `dq_out` e `dq_oe` foi essencial para garantir que o controlador só dirija o barramento no instante de escrita, eliminando risco de contenção.

5.3 Integração interface-controlador

A leitura automática após escrita foi o comportamento de integração mais relevante para a demonstração final. Esse mecanismo tornou a confirmação de dado imediata e reduziu erros operacionais durante os testes práticos.